Université Mohamed 1er
Faculté des Sciences
Département d'informatique

Filière SMI Semestre 6

TP
Module : Bases de données avancées

Années universitaires 2019/2020

Professeur : M. MOUSSI

TP1

Introduction

SQLPLUS est l'outil d'Oracle permettant l'utilisation du langage SQL. Cependant il offre en plus diverses commandes de manipulation de commandes SQL et de commandes propres à SQLPLUS.

Lancé l'interface SQLPLUS à partir du :
Démarrer ➔ Tous les programmes ➔ Oracle-OraHome90 ➔ Application Développement ➔ SQL Plus

Une boite de dialogue s'affiche pour demander le Login, Mot de passe et la chaine hôte.
La chaine hôte est le nom du serveur et le nom de la base de données. Ici on travaille localement, la chaine hôte se résume au nom de la base de données : **BD**.
Pour la première connexion donnez le login **scott**, le mot de passe **tiger** et chaîne hôte **BD.**

Une fois la connexion avec Oracle établie, SQLPLUS affiche son message d'attente :
SQL> <--------- indiquant qu'il attend une commande SQL

Les commandes SQL permettent de créer, mettre à jour et exploiter les tables de données.
Exemple : *SQL> select * from nom_table ;*

Les commandes SQLPLUS permettent de manipuler l'environnement dans lequel vont s'exécuter les commandes SQL.
Exemple : *SQL> describe nom_table*

Règles d'écriture des commandes SQL :

Elles peuvent s'écrire indifféremment en majuscules ou minuscules.
Une commande SQL se termine par " **;** " ou " / " ou une ligne blanche.
Le caractère " ; " indique la fin de la commande et demande son exécution
Le caractère " / " idem à au caractère ";" mais doit être seul sur sa ligne.
Ligne blanche termine la commande sans lancer son exécution

Soit le schéma relationnel des évaluations des étudiants qui contient les tables **EtudiantX**, **MatiereX** et **EvaluationX** (X est le nom de votre groupe ou binôme) décrites comme suit :

EtudiantX :
(**NE** : Numéro Etudiant de type number(4),
Nom : Nom Etudiant de type char (30),
Adresse : Adresse Etudiant de type varchar2(50),
Ville : Ville Etudiant de type char (15))

MatiereX :
(**NM** : Numéro Matière de type char(6),
Nom : Nom Matière de type char (15),
Coefficient : de type number(2)

EvaluationX :
(#**NE** : Numéro Etudiant de type number(4),
#**NM** : Numéro Matière de type char(6),
DateEval : date d'évaluation de type date,
Note : Note obtenue de type number(4,2))

Connexion :

Connectez-vous en utilisant le compte scott.

Création des tables :

Créer les trois tables de cette base de données en spécifiant les contraintes d'intégrité éventuelles (clés primaires, clés étrangères, champs non nuls…)

Manipulation des données :

1) Insérer des lignes dans chaque table.
2) Afficher tous les étudiants.
3) Afficher les étudiants ordonnés par ordre décroissant sur les villes.
4) Afficher sans redondance les villes des étudiants.
5) Afficher les étudiants habitants une ville donnée.
6) Afficher les évaluations ordonnées par date croissante.
7) Afficher toutes les matières ordonnées par ordre croissant sur les coefficients.
8) Insérer d'autres lignes dans les tables.
9) Modifier l'adresse et la ville de l'étudiant numéro 1.
10) Modifier la note de l'étudiant numéro 2 dans la matière numéro 3.
11) Ajouter 15% à la note de la matière numéro 2 pour tous les étudiants.
12) Supprimer les évaluations concernant la matière numéro 3.
13) Supprimer les évaluations pour lesquelles la note est inférieure à 5.
14) Afficher les numéros, les noms des étudiants et leurs notes.
15) Afficher les étudiants et leurs notes dans la matière numéro 5.
16) Afficher les notes de l'étudiant numéro 1 dans la matière informatique.
17) Afficher les notes des étudiants habitant une ville donnée dans la matière informatique.
18) Afficher le nom de chaque étudiant, le nom de chaque matière ainsi que les notes qu'a obtenue cet étudiant dans cette matière, ordonnées par notes décroissantes.
19) Afficher les noms des étudiants ayant obtenu la note la plus haute ou la note la plus basse.
20) Afficher pour chaque matière le nombre d'étudiants ayant obtenu la moyenne (note≥10).
21) Afficher pour chaque étudiant le nombre de matières dans laquelle il a eu sa moyenne.
22) Afficher la moyenne de chaque étudiant.

TP2

Soit la base de données relationnelle des évaluations des étudiants qui contient les tables **EtudiantX**, **MatiereX** et **EvaluationX** décrites dans le TP1 :

<u>Langage de contrôle de données :</u>

1) Connectez-vous en utilisant le compte *scott* et créez un utilisateur si possible. Sinon, en utilisant le compte système, modifier le compte *scott* pour qu'il puisse créer des utilisateurs (ajouter des privilèges).
2) Connectez-vous à nouveau en utilisant le compte *scott*.
3) Créer un utilisateur avec un nom et un mot de passe.
4) Modifier le mot de passe de cet utilisateur.
5) Donner le privilège de création de tablespace à l'utilisateur scott.
6) En utilisant scott, créer un *tablespace* nommé ***t_space_smiX*.**
7) Ajouter un fichier au *tablespace* créé.
8) Redimensionner un fichier ajouté.
9) Créer un utilisateur et lui affecter ***t_space_smiX*** par défaut et ***temp*** comme tablespace temporaire.
10) En utilisant le nouveau utilisateur (user1), créer les tables **EtudiantX**, **MatiereX** et **EvaluationX**
11) Utiliser *scott* pour consulter la table ***user1.EtudiantX***, conclure ?
12) Ce qui s'est passé avec *scott* est-il identique avec le compte system ?
13) Donner le droit de sélection sur la table ***user1.EtudiantX*** à l'utilisateur *scott.*
14) Créer un autre utilisateur 'utilisateur2'
15) Créer une autre table nommée **EtudiantX,** en utilisant le compte de l'utilisateur2, conclure ?
16) Supprimer le compte utilisateur2 puis vérifier l'existence de la table créée par cet utilisateur ?
17) Retirer le droit de création des utilisateurs du compte *scott*. Puis en l'utilisant essayer de créer un utilisateur.

Création et utilisation des vues et des index :

1) Connectez-vous en utilisant votre compte.
2) Créer une vue sur les étudiants habitant une ville donnée.
3) Utiliser cette vue pour faire des insertions, des affichages, des modifications et des suppressions.
4) Supprimer cette vue. Quel est l'effet de la suppression sur la table EtudiantX ?
5) Refaire les questions 2) en utilisant l'option *WITH CHECK OPTION* et la question 3)
6) Créer une vue sur les étudiants et les matières dans lesquelles ces étudiants n'ont pas obtenu la moyenne (Note < 10), de telle sorte à ce que cette condition soit toujours respectée.
7) Tenter de faire des insertions à travers cette vue.
8) Créer une vue comportant les champs : le numéro de l'étudiant, son nom et sa moyenne générale.
9) Utiliser cette vue pour faire des affichages, des insertions et des modifications.
10) Créer une vue comportant les champs : le nom et le coefficient de la matière.
11) Utiliser cette vue pour faire des affichages, des insertions et des modifications.
12) Créer et supprimer des index sur les trois tables.

TP3

Soit le schéma relationnel qui contient les tables **Etudiant**, **Matiere** et **Evaluation** décrites dans le TP1.

Exercice 1.

Ecrire un programme PL/SQL qui permet de

- saisir un certain numéro d'étudiant.
- Vérifie la note de cet étudiant dans la matière numéro 1 (ou autre numéro) : si cette note est inférieur à 10, elle sera augmentée par 2 sinon elle sera augmentée par 1.

Exercice 2.

Ecrire un programme PL/SQL qui permet de

- saisir un certain numéro de matière.
- Afficher le nombre d'évaluations dans cette matière.

Exercice 3.

Ecrire un programme PL/SQL qui permet d'afficher les entiers pairs inférieurs à 20.

Exercice 4.

Ecrire un programme PL/SQL qui permet de

- Saisir un entier n,
- Affiche les entiers pairs et les entiers impairs qui sont inférieurs à n.

Exercice 5.

Ecrire un programme PL/SQL qui permet de calculer et afficher la somme des entiers inférieurs à 10.

Exercice 6.

Ecrire un programme qui permet de tester les entiers inférieurs à 36 :

- Si l'entier est un multiple de trois, il affiche le message ' est un multiple de trois ' .
- Si l'entier est un multiple de quatre, il affiche le message ' est un multiple de quatre ' .
- Si l'entier est un multiple de cinq, il affiche le message ' est un multiple de cinq ' .

Exercice 7.

Refaire l'Exercice 1 en utilisant les curseurs.

Solutions TP1

Création des tables:

```
create table etudiant(NE number(6) primary key,nom_etud char(30),adresse varchar2(50));

create table matiere(NM number(3) primary key,nom_mat char(15),coefficient number(2));

create table evaluation( NE number(6) references etudiant(NE),NM number(3) references
matiere(NM),dateEval date,note number(4,2));
```

Manipulation des données:

1)

```
insert into etudiant values(1,'mohammed','fso');
insert into etudiant values(2,'ahmed','oujda');
insert into etudiant values(3,'aicha','rabat');

insert into matiere values(1,'math',4);
insert into matiere values(2,'info',3);
insert into matiere values(3,'phys',2);

insert into evaluation values(1,1,'10-10-2012',14);
insert into evaluation values(2,1,'01-02-2011',13);
insert into evaluation values(3,2,'03-03-2009',10);
```

2) afficher tous les etudiants

```
select * from etudiant
```

3) afficher les etudiants ordonnés par ordre décroissant sur les villes

```
select *
 from etudiant
 order by ville desc
--on a utilisé le numéro de la colonne ville
select *
from etudiant
ORDER BY 4 desc
```

4) Afficher sans redondance les villes des étudiants

```
select distinct ville
from etudiant
```

5) Afficher les étudiants habitants une ville donnée

```
select *
from etudiant
where ville='oujda'
```

6) Afficher les évaluations ordonnées par date croissante

```
select *
from evaluation
```

order by 3 asc

7) Afficher toutes les matières ordonnées par ordre croissant sur les coefficients.

```
select top 5 *
from matiere
order by Coefficient
--ou bien
select * from matiere
order by coefficient
```

8) Insérer d'autres lignes dans les tables

```
insert into etudiant values(5,'malki','28 koulouj','taouirirt');
insert into etudiant values(6,'ramdani','14 bd 14','fes');
insert into matiere values(6,'biologie',3);
insert into matiere values(7,'arch',4);
insert into evaluation values(4,2,'11/20/2000',12);
insert into evaluation values(4,1,'12/22/2000',12);
```

9) Modifier l'adresse et la ville de l'étudiant numéro 1

```
update etudiant
set Adresse='55 rue sahara'
where NE=1
```

10) Modifier la note de l'étudiant numéro 1 dans la matière numéro 3, la nouvelle note est 16

```
update evaluation
set Note=16
where NE=1 and NM=3
```

11) Ajouter 15% a la note de la matière numéro 2 pour tous les étudiants

```
update evaluation
set Note=Note+(0.15)*Note
where NM=2
```

12) Supprimer les évaluations concernant la note numéro 3

```
delete from evaluation
where NM=3
```

13) Supprimer les évaluations pour lesquelles la note est inférieure à 5

```
delete from evaluation
where Note<=5
```

14) Afficher les numéros, les noms des étudiants et leurs notes

```
select a.NE,b.Nom,a.Note
from evaluation a join etudiant b
on a.NE=b.NE
```

15) Afficher les étudiants et leurs notes dans la matière numéro 2

```
select a.NE,b.Nom,a.Note as "note de la matiere 2"
from evaluation a join etudiant b
```

```
on a.NE=b.NE
where a.NM=2
select * from evaluation join etudiant
```

16) Afficher les notes de l'étudiant numéro 1 dans la matière informatique

```
select a.Note  as "note d'informatique de l'eudiant N1"
from evaluation a join matiere b
on a.NM=b.NM
where
a.NE=1 and b.Nom='informatique'
    --ou bien
select Note as "note d'informatique de l'eudiant N1" from evaluation
where
NE=1 and NM in (select NM from matiere where Nom='informatique')
```

17) Afficher les notes des étudiants habitant une ville donnée dans la matière informatique

```
select Note as "note d'informatique des eudiants de oujda"
from evaluation
where NE in (select NE from etudiant where ville='oujda')
and NM in (select NM from matiere where Nom='informatique')
        --ou bien
select  a.Note as "note d'informatique des eudiants de oujda"
from evaluation a join etudiant b
 on  a.NE=b.NE
join matiere c on a.NM=c.NM
where b.ville='oujda'
and c.Nom='informatique'
         --ou bien
select a.note from evaluation a join matiere b on a.NM=b.NM
where b.Nom='informatique'
and a.NE in(select NE from etudiant where ville='oujda')
```

18) Afficher le nom de chaque étudiant, le nom de chaque matière ainsi que la note que la note qu'a obtenue cet étudiant dans cette matière, ordonnées par notes décroissantes*/

```
select  a.Nom as "Nom Etudiant",  b.Nom as "Matière", c.Note
from evaluation c join etudiant a
 on  c.NE=a.NE
join matiere b on c.NM=b.NM
order by c.Note desc
```

19) Afficher les noms des étudiants ayant obtenu la note la plus haute ou la note la plus basse.*/

```
select a.Nom
from etudiant a
join evaluation b on a.NE=b.NE
where
b.Note=(select max(Note)from evaluation) or b.Note=(select min(Note) from evaluation )
          --ou bien
select a.Nom
from etudiant a
```

```
join evaluation b on a.NE=b.NE
where
b.Note=(select max(Note)from evaluation)
union
select a.Nom
from etudiant a
join evaluation b on a.NE=b.NE
where b.Note=(select min(Note) from evaluation )
```

20) Afficher pour chaque matière le nombre d'étudiants ayant obtenu la moyenne (note>=10)

```
select a.Nom, count(b.NE) as "nombre d'etudiants ayant obtenu la moyenne"
from  matiere a join evaluation b
on a.NM=b.NM
where b.Note>=10
group by a.Nom

--pour afficher aussi le numéro de la matiere, on a :
select b.NM,
(select a.Nom from matiere a where a.NM=b.NM )as "matiere",
count(b.NE) as "nombre d'etudiants ayant obtenu la moyenne"
from evaluation b
where b.Note>=10
 group by b.NM
```

21) Afficher pour chaque étudiant le nombre de matières dans laquelle il a eu sa moyenne

```
 /*comment j'affiche aussi  le numéro de l'etudiant*/
select  b.NE,(select a.Nom from etudiant a where a.NE=b.NE)as "Nom etudiant",
 count(b.NM) as "nombre de matiere validées"
from  evaluation b
where b.Note>=10
group by b.NE
```

22) Afficher la moyenne de chaque étudiant.

```
select a.Nom, (sum(b.Note*c.Coefficient))/sum(c.Coefficient) as moyenne
from etudiant a,evaluation b,matiere c
where a.NE=b.NE
and b.NM=c.NM
group by a.Nom
```

Solutions TP2

Langage de contrôle de données :

1) Donner le privilège de création des utilisateurs
   Grant create user to scott

3) Création d'un utilisateur
   Create user smi_1 identified by smi
   Default tablespace users
   Quota unlimited on users
   Temporary tablespace temp

4) *Alter user smi_1*
   Identified by smi2

6) Creation d'un table space
   Create tablespace t_space_smi
   Datafile 'D : \fichier1.dbf' size 5M,
   'D : \fichier2.dbf' size 5M
   Minimum Extent 100 k
   Default storage (initial 100k next 100k [Maxextents 100] pct increase 0);

7) *Alter tablespace t_space_smi*
   Add datafile 'D : \fichier1.dbf' size 10M ;

8) *Alter database*
   datafile 'D : \fichier1.dbf' resize 20M ;

13) *Grant select on user1.eudiant to scott;*

16) *Drop user utilisateur2*

17) *Revoke create user from scott*

Creation et utilization des vues et des index

2) créer une vue sur les étudiants habitant une ville donnée

```
create view vue_ville
as
select * from etudiant
where ville='oujda'
```

3) Utiliser cette vue pour faire des insertions, des affichages, des modifications et des suppressions.*/

```
insert into vue_ville values(10,'fahmi','rue cas 20','rabat');
--remarque il va pas l'inserer dans la table vue mais dans la table etudaint car ville!='oujda'
```

```
insert into vue_ville values(11,'fahmi','rue cas 20','oujda');

update vue_ville
set nom='tahri'
where NE=4
--remerque : il va pas faire la mise à jour car ville!='oujda'

delete from vue_ville
where Nom like 'H%'
```

6) Créer une vue sur les étudiants et les matières dans lesquelles ces étudiants n'ont pas obtenu la moyenne (Note < 10), de telle sorte a ce que cette condition soit toujours respectées*/

```
create view vue_notefaible
as
select  c.NE,a.Nom,b.Nom as matiere,c.Note
 from
evaluation c join etudiant a on c.NE=a.NE
join matiere b on c.NM=b.NM
where c.Note<10
with check option

create view vue_etudiant
as
select * from etudiant
where
Nom like 'H%'
with check option
```

7) Tenter d'insérer a travers cette vue un étudiant et une matière ne respectant pas cette condition*/

```
insert into vue_notefaible values(1,'hssayni','chimie',15);
--il va pas faire l'insertion car la vue est sur plusieurs tables
--with check option : permet de faire l'insertion dans la table et non pas dans la vue si la condition n'est pas vérifiée

insert into etudiant values(13,'hamdi','22 rue 5','fes');

select * from etudiant
select * from vue_etudiant
```

8) Créer une vue comportant les champs : le numéro de l'étudiant, son nom et sa moyenne générale*/

```
create view vue_moyenne
as
select b.NE, (select a.Nom from etudiant a where a.NE=b.NE) as Nom,
 (sum(b.Note*c.Coefficient))/sum(c.Coefficient) as moyenne
from evaluation b,matiere c
where b.NM=c.NM
group by b.NE
```

9) Utiliser cette vue pour faire des affichage

```
select * from vue_moyenne
```

```
select moyenne from vue_moyenne
where Nom='tahri'
```

12)
```
CREATE UNIQUE INDEX  ix_etUd
ON etudiant (nom_etud);

DROP INDEX   ix_etUd ;
```

Solutions TP3

Exercice 1.

```
accept numero
DECLARE
num evaluation.NE%TYPE:=&numero;
note_etud evaluation.note%TYPE;
BEGIN
SELECT note into note_etud FROM evaluation
WHERE evaluation.NE=num
and evaluation.NM=1;
IF note_etud<10 THEN
  UPDATE evaluation
  SET note=note+1
  where NE=num;
ELSE
  UPDATE evaluation
  SET note=note+2
  where NE=num;
END IF;
END;
/
```

Exercice 2.

```
accept numero
DECLARE
nombre NUMBER;
num_mat evaluation.NM%TYPE:=&numero;
BEGIN
SELECT count(*) into nombre FROM evaluation
WHERE evaluation.NM=num_mat;
DBMS_OUTPUT.PUT_LINE(nombre);
END;
/
```

Exercice 3-4

```
accept nombre
DECLARE
n integer:=&nombre;
BEGIN
for i in 1..n LOOP
IF mod(i,2)=1 THEN
DBMS_OUTPUT.PUT_LINE(i|| 'est impaire');
ELSE
DBMS_OUTPUT.PUT_LINE(i|| 'est paire');
END IF;
END LOOP;
END;
```

Exercice 5

```
DECLARE
somme number:=0;
BEGIN
for i in 1..10 LOOP
somme:=somme+i;
END LOOP;
DBMS_OUTPUT.PUT_LINE('la somme de 1 a 10 est '|| somme);
END;
```

Exercice 6

```
DECLARE
n number;
BEGIN
for n in 1..36 LOOP
if mod(n,3)=0 THEN
  DBMS_OUTPUT.PUT_LINE(n || 'est un multiple de trois');
ELSIF mod(n,4)=0 THEN
  DBMS_OUTPUT.PUT_LINE(n || 'est un multiple de quatre');
ELSIF mod(n,5)=0 THEN
  DBMS_OUTPUT.PUT_LINE(n || 'est un multiple de cinq');
END IF;
END LOOP;
END;
/
```

Exercice 7(avec la saisie d'un numero)

```
accept numero
DECLARE
num evaluation.NE%TYPE:=&numero;
note_etud evalualuation.note%TYPE;
CURSOR curseur IS SELECT note FROM evaluation
WHERE evaluation.NE=num
and evaluation.NM=1;
BEGIN
OPEN curseur;
FETCH curseur into note_etud;
IF note_etud<10 THEN
  UPDATE evaluation
  SET note=note+1
  where NE=num;
ELSE
  UPDATE evaluation
  SET note=note+2
  where NE=num;
END IF;
CLOSE curseur;
END;
/
```

Exercice 7(avec la saisie d'un nom)

```
accept nom
DECLARE
name etudiant.nom_etud%TYPE:='&nom';
CURSOR curseur IS SELECT etudiant.NE,note FROM evaluation, etudiant
WHERE etudiant.nom_etud=name
and evaluation.NE=etudiant.NE
and evaluation.NM=1;
BEGIN
FOR enreg in curseur LOOP
IF note_etud<10 THEN
  UPDATE evaluation
  SET note=note+1
  where NE=enreg.NE;
ELSE
  UPDATE evaluation
  SET note=note+2
  where NE=enreg.NE;
END IF;
END LOOP;
END;
/
```